Home Content Contact

C++ STRUCTURE

FUNCTIONS, RETURN TYPE, VOID

Start

INTRODUCTION TO FUNCTIONS

- A function is a reusable sequence of statements
- It is designed to do a specific job
- Functions help avoid repeating code
- Every C++ program must have a function called main()
  - Program execution starts in main()

WHY DO WE USE FUNCTIONS?

- As programs grow, putting all code in main() becomes hard to manage
- Functions allow us to:
  - Break programs into smaller, modular parts
  - Make code easier to read, test, and maintain
- Programs usually contain many functions
- C++ provides built-in functions (**standard library**)
- Functions written by the programmer are called **user-defined functions**

HOW FUNCTION CALLS WORK

- A program normally runs statements sequentially
- When a function call occurs:
  - Execution of the current function is paused
  - The CPU “bookmarks” the current location
  - The called function runs
  - When it finishes, execution returns to the bookmark

C++ syntax

Home **Content** Contact

- Similar to:
  a. Pausing a book
  b. Making a phone call
  c. Returning to the exact spot you left off

EXAMPLE CODE

USER DEFINED FUNCTION

Functions that you write yourself are called **user-defined functions**.

```
returnType functionName()
{
     do something bla bla
    }
```

PARTS OF FUNCTION

- function header:
  - return type
  - function name
- function body
  - sequence of statements
  - the action the function does

WHY DO FUNCTIONS RETURN VALUES?

- Programs often have separate tasks
  - Get input
  - Process data
- Functions should be able to send results back to the caller
- Return values allow data flow between functions

FUNCTION RETURN VALUES (VALUE-RETURNING FUNCTIONS)

- A function that returns a value is called a value-returning function
- Its return type is not void
- The return type specifies what kind of value is returned

General form:

- Return type
- Function name
- Return statement inside the function

THE RETURN STATEMENT

- return sends one value back to the caller
- When return executes:
  - The value is produced
  - The function ends immediately
  - Control goes back to the caller

Important rules:

- Non-void functions must return a value
- Returning no value → undefined behavior

USING RETURN VALUES

- The caller can:
  - Store the returned value in a variable
  - Use it in an expression
  - Ignore it

Key point:

- Returned values are not printed automatically
- They are printed only if the caller prints them

MAIN() AND BEST PRACTICES

- main() is a value-returning function
- It must return an int
- Returning 0 means the program ran successfully

Best practices:

- Always return a value from non-void functions
- DONT REPEAT YOURSELF (DRY)
- Clearly define what a function’s return value means

VOID FUNCTIONS (NON-VALUE RETURNING FUNCTIONS)

- Void functions don’t need a return statement
- Void functions can’t be used in expression that require a value
- Returning a value from a void function is a compile error

SUMMARY

- Functions are a sequence of statements that is reusable
- Functions can return a value
- Functions must return the expected type
- Functions can be void, which doesn’t send anything back to the caller.

QUESTIONS?

ACTIVITY

1. In a whole sheet of paper
    a. code a function with
        i. return type is integer
        ii. function name is getIDnumber
        iii. when the function is called the user must input their id-number
    b. call the getInfo function inside main function
    c. then print it

ACTIVITY

2. Create a void function with a function name of readBook
    a. The readBook function must print “Currently Reading…”
    b. Now go back to the main function, in there first print “Opening book”
    c. after that, call the function readBook
    d. After it is called, print “Closing Book…”